



BIOINFORMATICS FOR THE LAB

From the Terminal to Biological Insight

A hands-on, beginner-friendly tutorial for biologists
with no coding experience required to get started.

What You'll Learn



Core command-line concepts

Get comfortable navigating and working in a terminal.



Files, programs & environments

Learn to manage and organize your project's files with confidence.



Common developer tools

See the tools bioinformaticians use every day, and why they matter.



Apply it to real lab work

Connect these skills to the analysis pipelines used in our lab.

The Bioinformatics Landscape



Omics

Genomics, transcriptomics, proteomics...



Phylogenetics

Studying evolutionary relationships



Structural Biology

Molecular & structural predictions

...and many more fields, each with its own specialized programs and tools.

What they all share in common

- Data formats, software installs, development environments & reproducibility
- This tutorial focuses on these transferable skills — useful across any lab or study



PART 1

Working in the Terminal

Learn the essential skills to move around, manage files, and run programs from the command line.

What Is a Terminal?

- A direct, text-based way to interact with your computer's file system
- **Why use one?**
 - Lightweight and efficient — no graphical program needed
 - Programmable — automate repetitive tasks
 - Helps you understand what's happening “under the hood”



Think of it as typing instructions directly to your computer, instead of clicking through menus.

Getting a Terminal on Your Computer



Mac & Linux

- Built-in Terminal app
- Ready to use — no setup needed
- Runs on Unix-based systems

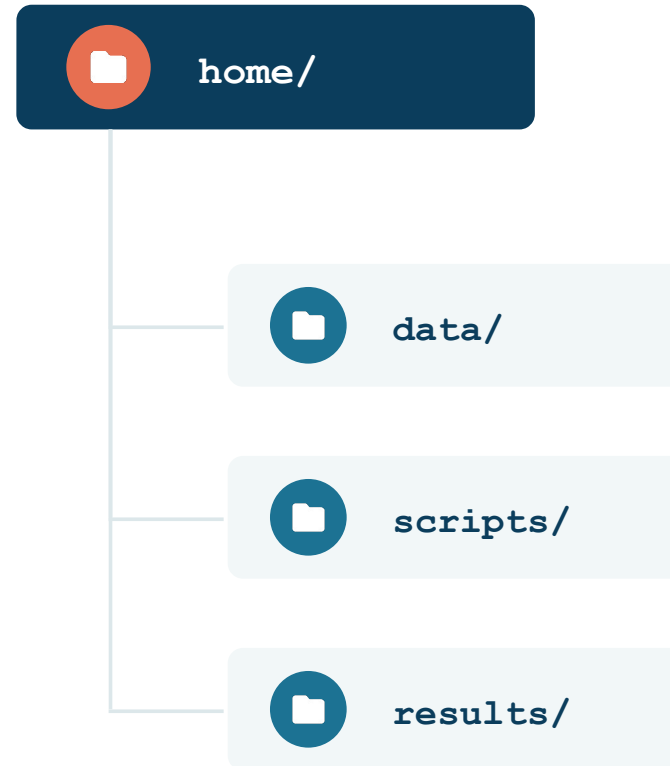


Windows

- Comes with PowerShell
- Best option: install WSL
- WSL = a full Linux terminal experience

Navigating Files & Directories

- Directories = folders that organize your files
- Every file/folder has a path — its address in the system
- Hidden files start with a dot, e.g. `.bashrc`
- **Tip: avoid spaces in file names — use `_` or `-`**



Core Shell Commands



pwd

Show current directory

```
$ pwd
```



ls

List files & folders

```
$ ls -l
```



cd

Change directory

```
$ cd data/
```



mkdir

Make a new folder

```
$ mkdir results/
```



cp

Copy a file

```
$ cp a.txt b.txt
```



mv

Move or rename a file

```
$ mv old.txt new.txt
```



rm

Delete — no undo, be careful!

```
$ rm temp.txt
```



--help

Show help for any command

```
$ ls --help
```

Text Editors & Helpful Tips



Terminal text editors

vim, emacs, and nano let you create and edit files without leaving the terminal.



Get help anytime

Add --help after most commands to see how they work.



Tab-completion

Press Tab to auto-complete file and folder names — faster, fewer typos.



Wildcards

Use * to match multiple files at once, e.g. *.fastq matches all FASTQ files.

Understanding File Permissions

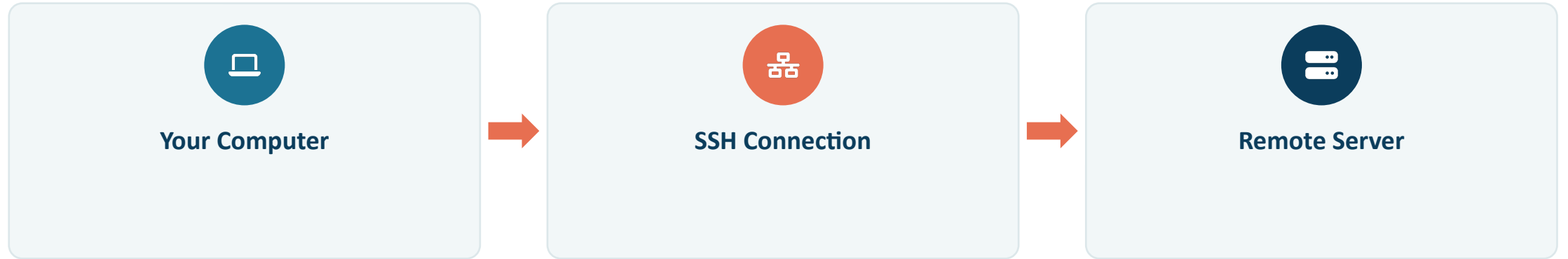
ls -l shows permissions like this for each file or folder:

d	rwx	r-x	r-x
Type	Owner	Group	Others
d = directory (- = file)	What you can do	What your group can do	What everyone else can do

r = read **w** = write **x** = execute (run)

You won't need to change permissions often — this is just so ls -l output makes sense.

Connecting to a Remote Computer



- Many labs run analyses on a shared remote computer (a server or HPC cluster)
- You'll need a username and password to connect
- Once connected, you use the same terminal commands as on your own computer

Connecting & Transferring Files

- ssh logs you into a remote computer — the password prompt shows nothing as you type (that's normal!)

```
# Log in to a remote computer
```

```
ssh user@remoteComputer.ca
```

```
# Copy a file TO the remote computer
```

```
scp file.txt user@remoteComputer.ca:remoteDirPath/
```

```
# Copy a file FROM the remote computer
```

```
scp user@remoteComputer.ca:remoteDirPath/file.txt localPath/
```

scp copies files between computers. rsync is a faster alternative for large transfers.

Combining Commands

- **Inspect & process text files:**
 - cat, head, tail, grep, wc — useful, everyday commands
- Pipe | sends the output of one command into the next
- > and >> direct output to a file
- Ctrl + C stops a running command or process

```
# Count how many lines in data.txt contain "gene1"  
grep "gene1" data.txt | wc -l
```

Two more powerful (but more advanced) tools you'll see in scripts: awk and sed, for pattern-based text editing. No need to master their syntax now — just recognize them.

Recap: Terminal Basics



Navigate locally

Move around and manage files with `cd`, `ls`, and `pwd`.



Connect remotely

Log into a shared computer with `ssh`.



Transfer files

Move files between computers with `scp` or `rsync`.



Inspect & combine commands

Use `grep` and pipes to search and process data files.



PART 2

Scripting & Programming

Turn manual, repeated steps into automated, reproducible workflows.

Why Automate Your Work?



Efficiency

Automate repetitive tasks instead of repeating them by hand.



Fewer mistakes

Removes human error from repeated steps.



Reproducibility

Anyone can rerun your exact steps and get the same result.



It's just a text file

A script (e.g. .sh) is a plain text file listing instructions. Note: "scripting," "coding," and "programming" all mean the same thing.

Anatomy of a Shell Script

```
#!/bin/bash  
echo "Starting analysis..."  
mkdir results/  
python analyze.py --input data.csv
```

This turns three manual steps into one command you can run anytime, on any computer.

- `#!/bin/bash` tells the computer this is a bash script
- `.sh` is the file extension for a shell script
- Each line runs in order, top to bottom
- Save it, then run it from the terminal

Choosing a Language: Bash, Python & R

Bash

- Native language of UNIX/Linux
- Great for chaining commands & files
- Best for simple, linear workflows
- Not ideal for complex logic

Python

- Popular, general-purpose language
- Huge library ecosystem
- Intuitive, readable syntax
- Great for automation & data wrangling

R

- Built for statistics & visualization
- Widely used in bioinformatics analysis
- Strong plotting capabilities
- Large collection of stats/bio packages

Writing & Running Code



Terminal editors

vim, nano, etc. work, but only text-based and can be slow for larger scripts.



IDEs (Integrated Development Environments)

PyCharm, Spyder, RStudio, VS Code — better interactive workflow, autocomplete & debugging.



Jupyter Notebooks

Mix code, output, and notes together — tell a clear narrative while staying flexible and reproducible.

Installing External Packages



Python

- Packages come from PyPI or conda-forge
- Install with:

```
pip install <package>
```

```
conda install <package>
```



R

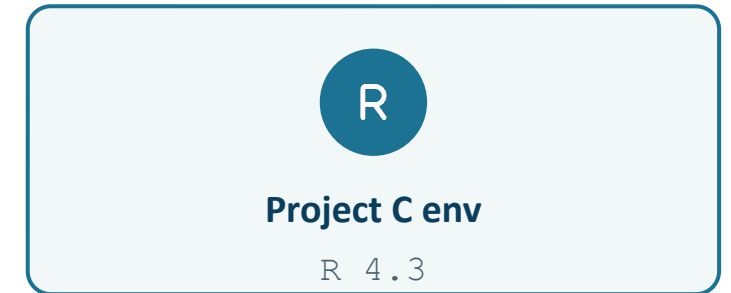
- Packages come from CRAN
- Install with:
- Bioinformatics packages often use BiocManager:

```
install.packages("pkg")
```

```
BiocManager::install("pkg")
```

Avoiding Conflicts with Environments

- Programs often depend on specific versions of other packages
- Installing many projects together can create version conflicts
- A package environment (e.g. conda, virtualenv) keeps each project's packages isolated



Isolated — no conflicts between projects

```
conda create --name myEnv python=3.11  # create the environment
conda activate myEnv                    # switch into it
conda install numpy pandas              # install packages inside it
conda deactivate                        # leave the environment when done
```

Recap: Scripting & Programming



Automate your work

Use shell, Python, or R scripts to run repeated tasks reliably.



Install packages

Pull trusted tools from PyPI/conda-forge (Python) or CRAN/Bioconductor (R).



Use environments

Keep each project's packages isolated to avoid conflicts.



Write & run code

Use an IDE, a notebook, or a terminal editor — whatever fits your task.



PART 3

Additional Software Tools

Tools that help you track changes, share environments, and manage complex workflows.

Version Control: Git & GitHub

● Git

- Tracks changes to code over time
- Lets you undo mistakes and see history

● GitHub

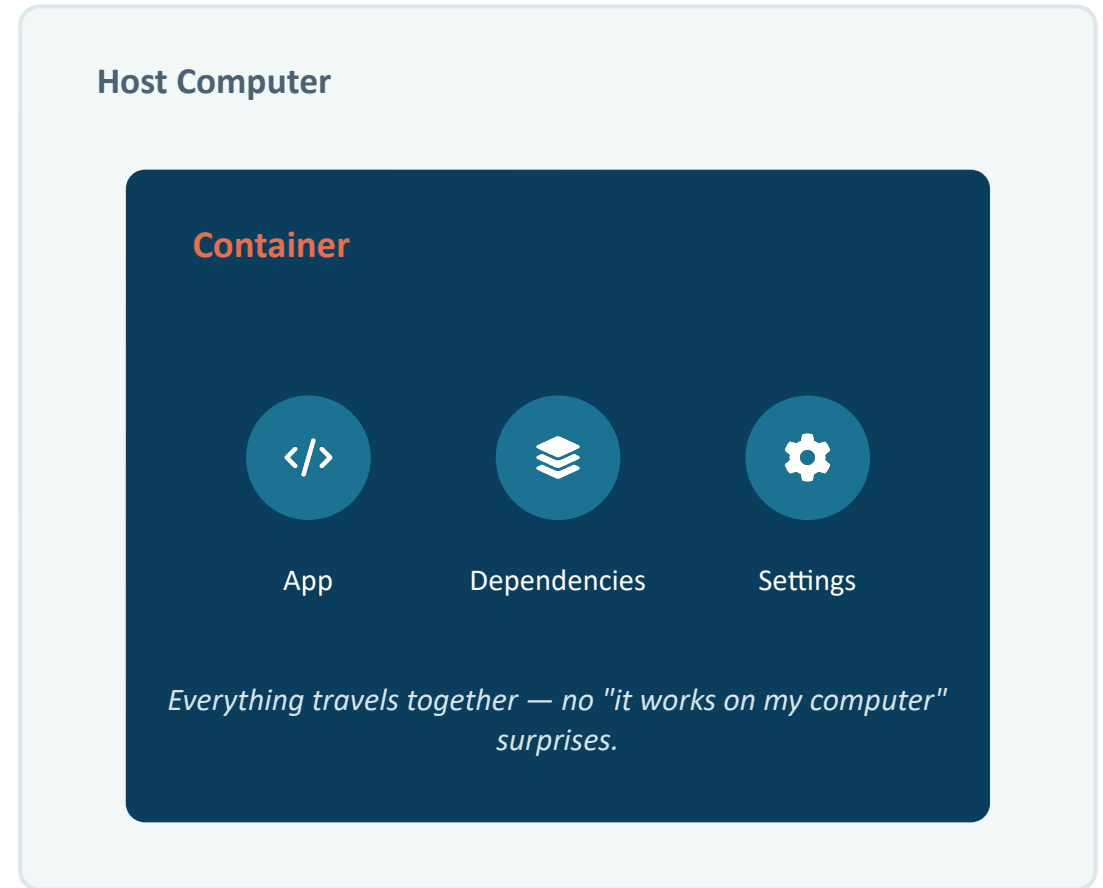
- Hosts code online (a "repository")
- Makes it easy to share & collaborate

```
# Download a repository
git clone
https://github.com/<repository>.git

# Get the latest updates
cd repository/ && git pull
```

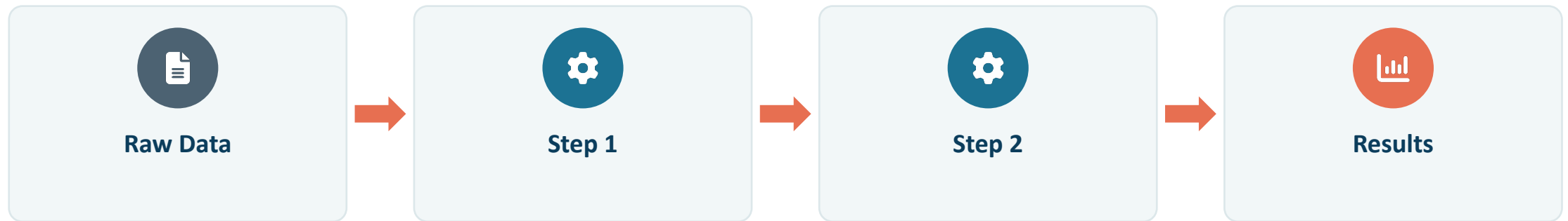
Containers: Docker & Apptainer

- A container packages software with everything it needs to run
- Keeps your analysis environment consistent across computers
- Docker is common; Apptainer (formerly Singularity) is common on shared lab/HPC servers



Orchestrating Workflows

- Bioinformatics analyses often involve many sequential steps
- Workflow managers like Snakemake and Nextflow automate the entire pipeline
- They handle step order, computing resources, and reruns for you



Managed automatically, in order, by the workflow tool

These Tools in Practice



Git

```
git clone https://github.com/<repository>.git
```



Apptainer

```
apptainer exec image.sif <program command>
```



Nextflow

```
nextflow run <pipeline>.nf
```

Recap: Additional Software Tools



Track changes with Git

Version your code, and share it with others via GitHub.



Keep software consistent with containers

Docker or Apptainer package an analysis environment so it runs the same everywhere.



Automate multi-step analyses

Snakemake or Nextflow run each step in order, handling resources and reruns.



Together, these make analyses reproducible

Anyone — including future you — can rerun the exact same pipeline and get the same result.



PART 4

Lab Bioinformatics Pipelines

Putting it all together: three real analysis pipelines used in the lab.

What's Ahead



Common file types

Get familiar with the file formats you'll see throughout these pipelines (FASTQ, BAM, BED, etc.).



Processing sequencing reads

Turn raw reads into clean, aligned data ready for analysis.



Differential binding analysis

Compare protein-DNA binding sites across conditions with DiffBind.



Differential expression analysis

Compare gene expression across conditions and identify meaningful changes.

Common File Types You'll Encounter



FASTQ (.fastq / .fq)

Raw sequencing reads plus per-base quality scores. This is the starting point of every pipeline.



BAM / SAM (.bam / .sam)

Sequencing reads aligned to a reference genome. BAM is the compressed, binary version of SAM.



BED (.bed)

Simple genomic intervals (chromosome, start, end) — often used for peaks or regions of interest.



Count matrix (.csv / .txt)

A table of read counts per gene or region, per sample — the input to statistical analysis.



Sequences (.fa / .fasta)

Format for genomes, proteomes, or any list of nucleotide or amino acid sequence.



Annotation (.gtf / .gff)

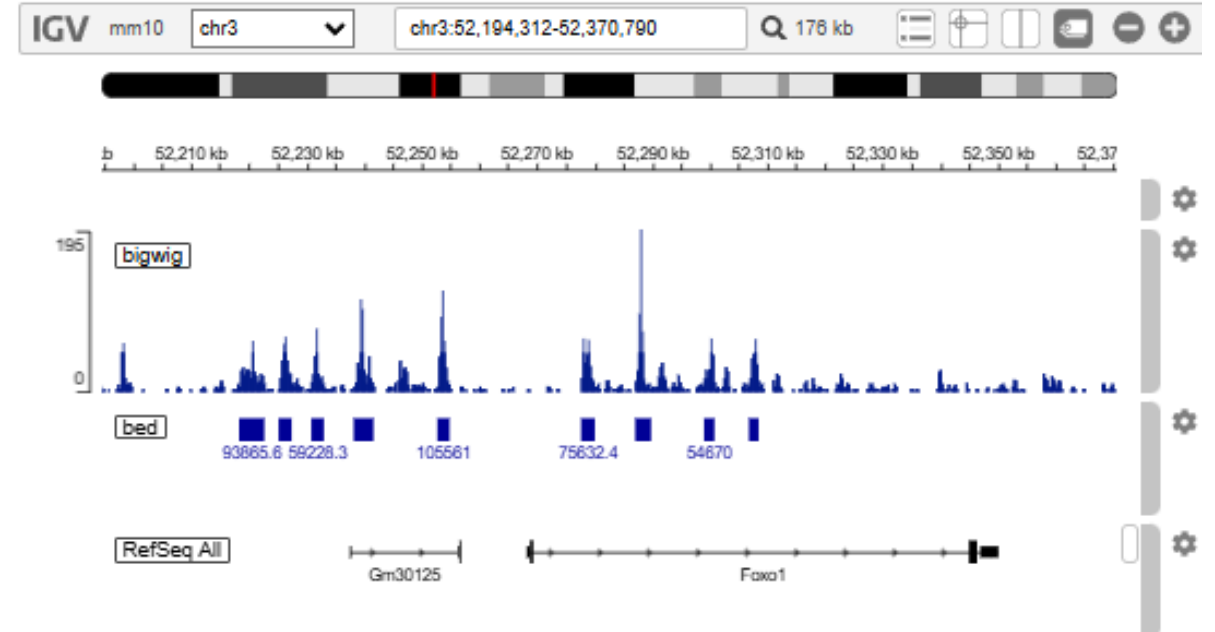
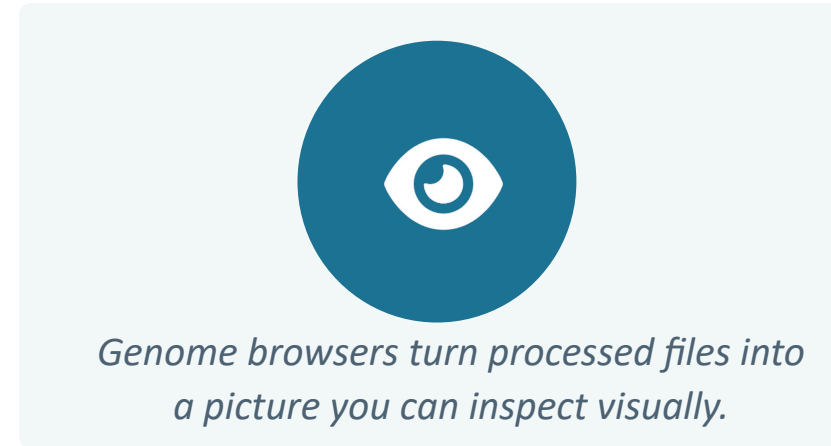
Describes where genes and features are located on the reference genome.

Note 1: some formats are binary (not human-readable) and need special programs to open.

Note 2: Files ending in .gz are simply compressed (like .zip); raw reads are commonly kept compressed to reduce file sizes.

Visualizing Sequencing Data

- Genome browsers let you see reads, peaks, and annotations lined up against the genome.
- Useful for sanity-checking your data before and after each analysis step.
- Load BIGWIG, BED, and other files into a viewer like this throughout the pipelines ahead.



Visualizing Sequencing Data



UCSC Genome Browser

- Hosted as a web application
- Huge library of built-in datasets & annotations
- Great for sharing a link to a specific view
- Can be slower to set up with many custom tracks
- Needs external data pointed to a database (e.g. CyVerse)

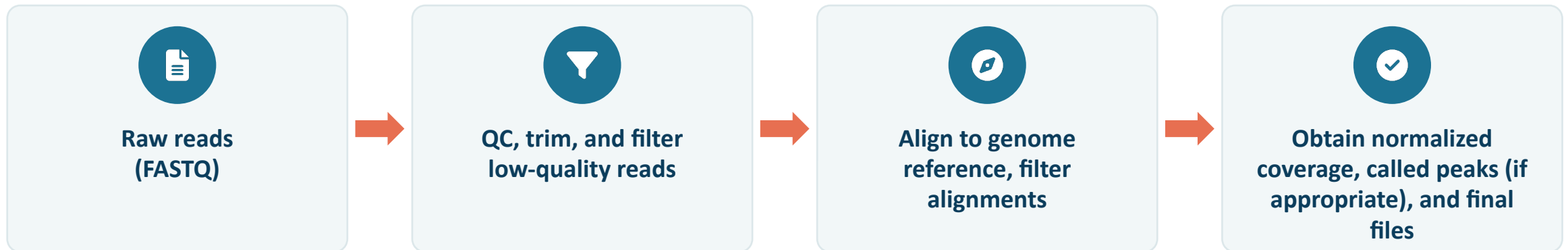


IGV (Integrative Genomics Viewer)

- Web app or desktop application
- Fewer built-in datasets & annotations
- Fast for quick, local inspections
- Good for spot-checking your own BAM/BigWig files
- Great before/after QC of your own data

Processing Sequencing Reads

- Many programs and steps required with various commands to process reads, so an automatic pipeline makes it much easier and repeatable
- Raw reads are cleaned, aligned, and processed according to their assay and experiment before any downstream biological analysis begins.



- Once you have processed data, you're ready for visualizing and downstream analysis.

Overview: Processing Reads Pipeline

3 Nextflow instruction files



`process_reads.nf`

Defines the pipeline



`modules.nf`

Defines each process/step



`process_reads.config`

Defines parameters & options



Input: raw sequencing reads (.fastq.gz)



Nextflow runs each step in order, using Apptainer for consistent software



Output files, ready for downstream analysis

Staging the Pipeline

- Clone the Nextflow pipeline from the lab GitHub or download the 3 files
- Download bowtie2/hisat2 genome index files if not present
- Setup your sample reads
- Prepare your Nextflow configuration and override default parameters
- Create and submit a batch script to the HPC job scheduler



Write once, submit, and let Nextflow and the scheduler run every step for you — no need to babysit the terminal.

Setting Up Your Inputs

Key parameters in the .config file to override defaults

target

control

assembly

assay

length

normalize_by

genome_index

reads_type

Review/change these before running the pipeline.

Organizing your read files



WT-1/

wt-1_R1.fastq.gz
wt-1_R2.fastq.gz



WT-2/

wt-2_R1.fastq.gz
wt-2_R2.fastq.gz

Paired-end reads stay together, named ..._R1 and ..._R2

Running Locally vs. on the HPC



Your Laptop

- Good for small test files and quick checks
- Limited memory and CPU power
- Ties up your own machine while running
- No job queue — runs immediately



Lab HPC Cluster

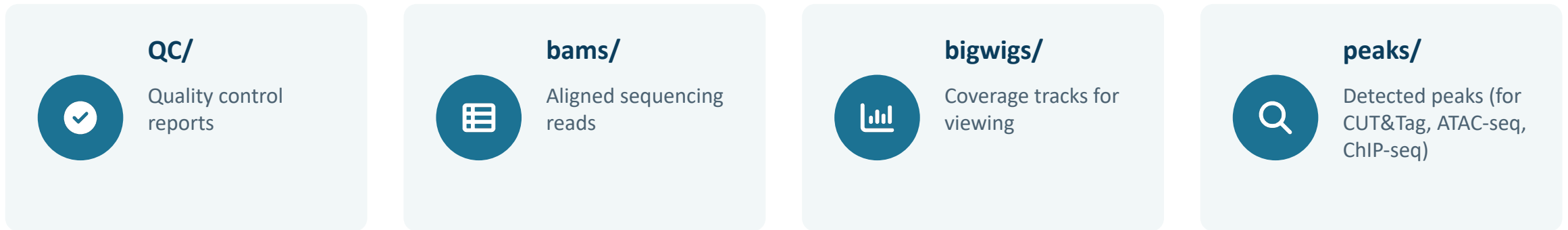
- Built for large sequencing datasets
- Many CPUs and large memory available
- Runs in the background via a job scheduler
- Shared resource — jobs may wait briefly in a queue

Example: Running the Pipeline

```
nextflow run process_reads.nf -c process_reads.config \  
  --target KO/ --control IgG/ \  
  --assay cutntag --assembly mm10
```

- --target / --control — which sample (director with reads) to process
- --assay — type of analysis (e.g. CUT&Tag, ChIP-seq, ATAC-seq)
- --assembly — reference genome build to use
- *See the .config file for the full list of adjustable options*

Output Files



- Each processing step produces new files using the lab Apptainer, so tools and versions stay consistent for everyone.
 - Intermediate process files can be found in the Nextflow working directory folder created on each run (defined by the `--workDir` parameter)
- Note: BAM files are essential and can be used to produce many other files (bigWigs, peaks, count matrices, etc.) and for downstream analysis.

Differential Binding Analysis: Overview

- Differential binding analysis compares where a protein binds DNA across different conditions (e.g. WT vs. KO).
 - Common in ChIP-seq and CUT&Tag experiments.
- Peaks (regions of enriched binding) are called for each sample, then compared across conditions.
- DiffBind is the R package the lab uses for this comparison step, wrapped in an executable script.



Goal: find binding sites that change significantly between conditions.

DiffBind Input: The Sample Sheet

- DiffBind needs a sample sheet describing every sample and where its files live, e.g. `diffbind_samplesheet.csv`:

SampleID	Condition	Replicate	Peaks	PeakCaller	bamReads
WT-SEACR-1	WT	1	WT1.bed	SEACR	WT1...MAPQ10.bam
WT-SEACR-2	WT	2	WT2.bed	SEACR	WT2...MAPQ10.bam
KO-SEACR-1	KO	1	KO1.bed	SEACR	KO1...MAPQ10.bam
KO-SEACR-2	KO	2	KO2.bed	SEACR	KO2...MAPQ10.bam

- Each row is one sample: its condition, replicate number, peak file, and aligned BAM reads.

How DiffBind Analyzes Your Data

- DiffBind compares your peak sets across conditions in two stages, run in order:

1 

Occupancy Analysis

- Overlaps peak calls across replicates & conditions
- Identifies consensus, unique & shared binding regions
- Can be visualized as a Venn diagram or PCA of peak overlap
- Answers: "Where is binding happening, and in which conditions?"



2



Affinity Analysis

- Counts aligned reads under each region from the BAM files
- Applies statistical testing (via DESeq2 or edgeR under the hood)
- Scores each region for fold-change & significance
- Answers: "How much does binding differ, and is it significant?"

```
apptainer exec image.sif Rscript analyze_peaks.R --file samplesheet.csv
```

How DiffBind Analyzes Your Data

- Reads in the sample sheet, then builds a consensus set of peaks seen across your samples.
- Counts how many aligned reads fall inside each peak, for every sample.
 - This gives a table of "binding scores" — similar in spirit to the count matrix used in gene expression.
- Runs a statistical test to find peaks whose binding score differs significantly between conditions.
- Outputs a ranked list of differentially bound regions, with significance values for each.

Differential Binding: What You Get



Differential binding table

Every consensus peak, its binding score per sample, and a significance value.



PCA / correlation plots

Visual check that replicates cluster together and conditions separate.



Volcano / MA plots

Highlight the peaks with the largest, most significant binding changes.

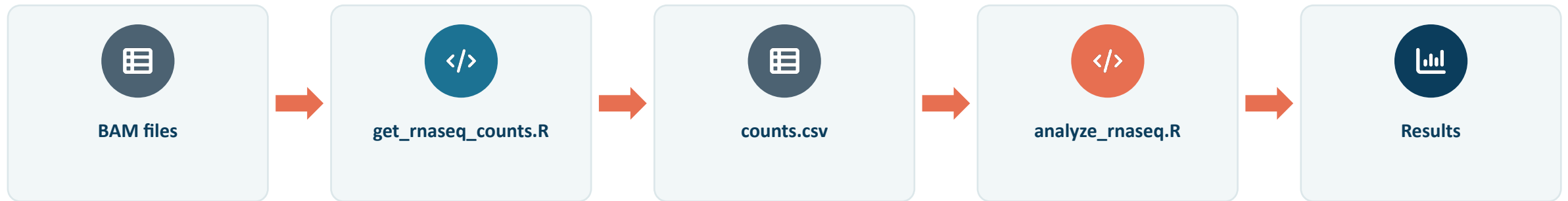
Differential Expression Analysis: Overview

- Differential expression analysis compares gene expression levels across conditions (e.g. Control vs. KO).
 - Common in RNA-seq experiments.
- Aligned reads are counted per gene to build a count matrix, then compared statistically across conditions.
- Tools like DESeq2 or edgeR (in R) are commonly used for this comparison step, and are wrapped in an executable script for the lab.



Goal: find genes whose expression changes significantly between conditions.

From BAM Files to Results



```
# Put all .bam and .bai files into a single folder, then:
apptainer exec image.sif Rscript get_rnaseq_counts.R --bams myBams/

# Sample column in sampleinfo file must match counts.csv sample names
apptainer exec image.sif Rscript analyze_rnaseq.R \
  --countsfile counts.csv --sampleinfo samples.csv
```

From BAM Files to Results

- Just like DiffBind, this pipeline needs a sample sheet describing every sample — here, `sampleinfo.csv`.
- Reads are counted per gene for each sample's BAM file, producing one column of counts per sample.
- These per-sample columns are combined into a single count matrix, then handed to DESeq2 or edgeR.
- Both the sample sheet and the count matrix must use the exact same sample names — this is what lets the software match each column of counts to the right condition.

These 2 Files Must Match

- **sampleinfo.csv** — describes each sample:

SampleID	Condition	Replicate	Batch
Control_1	Control	1	1
Control_2	Control	2	1
Control_3	Control	3	1
KO_1	KO	1	1
KO_2	KO	2	1
KO_3	KO	3	1

- **count matrix** — SampleID = column headers:

Gene	Control_1	Control_2	Control_3	KO_1	KO_2	KO_3
Gene_A	120	134	118	45	52	48
Gene_B	10	8	12	95	101	88
Gene_C	300	312	289	295	305	298

- Notice the SampleID column in sampleinfo.csv exactly matches the column names in the count matrix — this is required for the software to know which counts belong to which condition.

Differential Expression: What You Get

Differential expression table



Every gene, its fold-change between conditions, and a significance value.

PCA / clustering plots



Visual check that replicates cluster together and conditions separate.

Volcano plots



Highlight the genes with the largest, most significant expression changes.

Recap: Lab Bioinformatics Pipelines



Processing reads is the shared first step

Trim, align, and quality-check raw FASTQ reads into a clean BAM file using the lab Apptainer.



Differential binding finds where binding changes

DiffBind compares peak binding scores across conditions using a sample sheet like `diffbind_samplesheet.csv`.



Differential expression finds where genes change

DESeq2/edgeR compare a gene count matrix across conditions using a sample sheet like `sampleinfo.csv`.



Sample sheets and data files must always match

Consistent sample names are what let every one of these tools connect the right data to the right condition.



YOU'RE READY TO GET STARTED

From the Terminal to Biological Insight

Questions? Reach out to the lab computing team — we're happy to help you get set up.